A Private AI Assistant For The Lab

What we can build, how fast it will feel, and what it will cost

`libr-local-llm`

The idea, in one sentence

Put an AI assistant **on the lab's own computers**, so several people can use it at once — and so it is allowed to read patient data, which no outside service ever will be.

Today

You use Claude. It is excellent and fast. But every word you send leaves the building, so it can never touch a patient recording or transcript.

What this would add

An assistant living on our own machines. A bit slower. Not quite as clever. But it can look at the actual data, because nothing ever leaves.

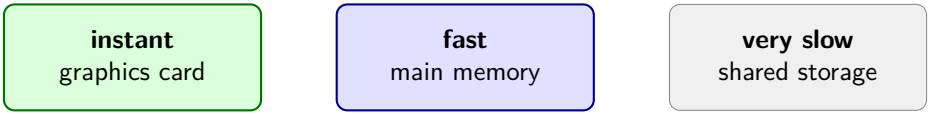
This is **not** a replacement for Claude. It is a second tool that can go where Claude is not allowed to go.

First, the one idea everything else depends on

The model is not one brain. It is about 19,000 narrow specialists.

One knows about Python. One knows about Italian grammar. One knows about chemistry names. Nobody knows everything.

To write each word, the model picks the handful of specialists that word needs and **reads their notes**. Reading the notes is the work; everything else is quick. So the design question is: *where do we keep 19,000 sets of notes?*



The reason this project is possible at all: our machines have enough main memory to hold *every one* of the 19,000 sets of notes. The assistant never has to go to slow storage while it is answering you.

Why it is slow: 600 specialists consulted, per word

Writing one word is not one lookup. The model works through the question in **75 rounds**, and in each round it consults **8** specialists.

$$75 \text{ rounds} \times 8 \text{ specialists} = \textbf{600 sets of notes read, for every single word.}$$

That is the entire speed problem, and no clever engineering removes it. **Reading notes is the work.** We can keep the notes somewhere faster — that is what the previous slide was about — but we cannot read fewer of them.

And reading is only *half* the job. The other half is the thinking the model does once it has the notes in front of it. So even if reading were free, we would only be about **twice** as fast. Worth knowing before anyone promises ten times.

What happens when several people ask at once

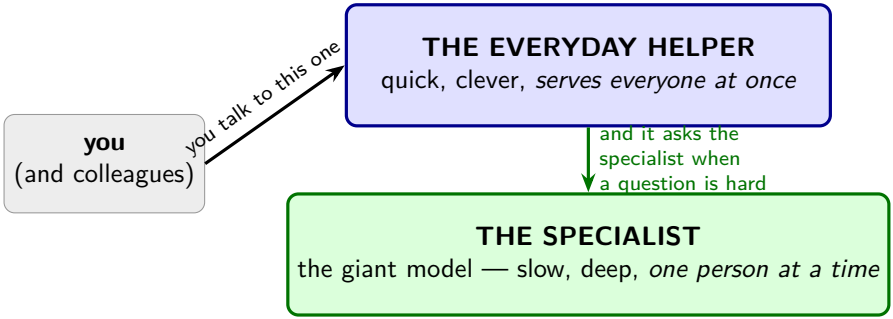
	your wait for a page
you, working alone	about 1 minute
you, with 3 colleagues also asking	about 2–3 minutes
you, with 7 colleagues also asking	**about 5 minutes**

Why: eight people need eight *different* specialists. The model cannot answer everyone by reading one set of notes — it has to read nearly everybody's. Sharing would only help if you all wanted the same specialists, and you don't.

Handling everyone together *does* help a little: the group gets through about **1.7 times** as much work as taking turns would. But that small gain is spread across eight people, while **your own answer arrives five times later.** Measured, not estimated.

The conclusion: a giant model can serve *one* person well, or a crowd badly. For a crowd you need a **smaller** model — which is why we build more than one.

So: three helpers, not one



Like a good office. You talk to the capable colleague who is always there. When something genuinely difficult comes up, *they* ask the world expert down the hall and come back with the answer.

A **third** helper runs overnight, grinding through big piles of paperwork — scoring thousands of transcripts. Nobody talks to it.

Helper 1 — the everyday one

How big	about a third the size of the specialist — still very large
Where it lives	our one machine with four graphics cards
How many people	**everybody, at the same time**
How fast	roughly **a page of text every 20–30 seconds**
What it is for	**almost everything you do all day**

Why this one can serve a crowd and the specialist cannot: it is small enough to live entirely on the graphics cards, which are enormously faster than the shelves. The book-fetching stops being the problem, so serving several people at once genuinely does work here.

We have not picked exactly which model yet — that is the first real job. It has to fit on the four cards, be good at code, and be able to use tools. **It is not the small model that was tried and rejected earlier**: this one has roughly eight times as many numbers in it and runs on much better software.

Helper 2 — the specialist

How big	**744 billion** — the biggest thing we can run at all
Where it lives	one ordinary machine, using most of its memory
How many people	**one at a time.** Others wait in line
How fast	roughly **a page of text per minute**
What it is for	the hard question you would otherwise be stuck on

One catch worth planning around. Waking it up means reading 372 gigabytes off the shared storage. That takes about **27 minutes**. So we wake it in the morning and leave it awake, rather than starting it on demand.

It is slow because it is enormous, and it is worth having *because* it is enormous. A minute of waiting for a genuinely hard answer is a good trade. A minute of waiting to rename a variable is not — which is exactly why helper 1 exists.

How good is the specialist, really?

The honest answer depends entirely on *what kind* of question you ask.

What you ask	Versus Claude	How long
"Here is a function and a bug report — what is wrong?"	**genuinely close**	1–4 minutes
"Which of these two designs is better, and why?"	**genuinely close**	1–4 minutes
"Read these eight files and find the inconsistency"	**not close**	several minutes before it even starts
"Keep working on this with me for an hour"	**don't**	see the next slide

One hard question, with everything it needs already in front of it. That is what the specialist is for, and there it is worth the wait. When the job becomes "hold a lot in your head and notice something subtle", the gap opens wide.

An awkward twist: the harder the question, the more it thinks before answering — and thinking is what it is slow at. The questions most worth asking are the ones you wait longest for.

“Can I just write down what I want and let it code?”

Yes — with the everyday helper, and with limits worth knowing first.

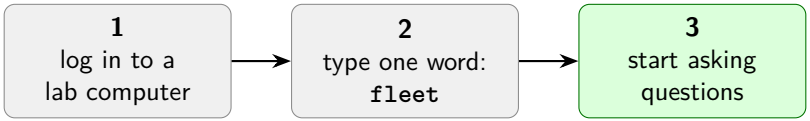
A moderate coding job, start to finish	**How long**	**Sensible?**
Claude, today	3–5 minutes	yes
the everyday helper	**6–12 minutes**	**yes**
the specialist	about 30 minutes	no — leave it alone

Two to four times slower than Claude for the same job. Write the description, start it, go and do something else for ten minutes. A real workflow, not a demo.

The catch is not about speed. To edit your files the model fills in a little form for each action, and because we run a *compressed* copy it **sometimes fills the form in wrong**. There is a built-in repair for the simple forms — open a file, run a command — but **not for the complicated ones**, and “change this line to that line” is a complicated one.

Start with small, clearly described jobs and check the result. How often it fills the form in wrong is one of the first things we measure — if it is often, “hand it a spec” is a demo, and we want to know that early.

What it looks like to use



That is the whole interface. One word. No settings, no flags, no remembering which helper to ask for. It starts anything that is not already running, waits for it, picks up your last conversation in that folder, and opens.

`fleet`	**everything.** This is the one you type
`fleet status`	is it running? is anyone waiting?
`fleet down`	switch it off now, rather than letting it idle off

You never ask for the specialist by name either. When a question is hard, the everyday helper goes and consults it, says so, and comes back with the answer — the same way you would ask a colleague to check something with an expert.

“Wait — do we need to pay for an API key?”

No. Nothing is bought, nothing is billed, and no word you type ever reaches Anthropic or anyone else outside the building.

There are two things called a “key” here, and only one is real.

The fake one	The Claude software refuses to start unless a certain box has *something* in it. So we type the word `local`. It is not an account, it is not checked by anyone, and it never leaves the machine. It is a seatbelt light, not a ticket.
The real one	**A password of our own**, that we choose, protecting our own assistant. This one matters.

Why a password at all: these are **shared** computers. Anyone else logged in could otherwise walk up and use our assistant — spending our share of the lab’s computing time, and asking it things we would be answerable for.

One specific hazard: conversations sit in numbered slots. If a stranger picks the number you are using, you do not get two conversations — you get one, tangled together.

Being a good neighbour

These are shared computers, and our assistant would sit on a fair chunk of them all day. **Every part of sharing them politely is automatic.** You do nothing.

The subtle one is "waits". Without it the system hands a computer back, notices a second later that it is short of computers, and grabs the same one straight back — beating the very colleague it was trying to help. It would look rude while running code written to be polite.

notices a colleague is stuck	**automatic**
finishes the answer you are waiting on first	**automatic** — it never cuts you off mid-sentence
hands the computer back, waits, takes it again later	**automatic**
switches off when unused, back on when needed	**automatic**
restarts itself after a crash	**automatic**
reconnects you when it moves to a different computer	**automatic**
holds your question while it is starting up	**automatic** — you see "starting", not an error
switch it off right now	**the only button you ever press**

How we will know whether it is any good

Speed we can measure. **Whether it is *smart* enough is harder** — and we have an unusually good way to test it.

Use this project as the exam. Planning this system turned up three real discoveries about our own hardware that nobody had noticed. We know the answers now. Hand the local assistant the same starting point and see how many it finds on its own.

Would we bet on it?	The discovery it has to make
maybe	one of our job queues can silently freeze the assistant mid-answer
probably	asking for memory quietly charges us for twice the computers
probably not	**why serving eight people at once cannot work**

Grade this hardest: does it *hold its ground* when told it is wrong? One that folds and invents a better-sounding answer is worse than one that never spotted the problem.

What to expect: six people log in on Monday morning

Question	Honest answer
Can six people use it at once?	**Yes.** All six talk to the everyday helper, all at the same time, and it is built for exactly that.
How fast for each of them?	**A page of text in 20–30 seconds.** It does not noticeably slow down as colleagues join.
How does that compare to Claude?	Claude does that page in about **9 seconds**. So: two to three times slower. **Noticeable. Not painful.**
What if two people need the specialist?	They **queue**. It serves one at a time, about a minute a page, so the second person waits about two minutes.
First thing Monday?	The specialist needs **27 minutes** to wake up. We wake it before people arrive.
Will it be as clever as Claude?	**For everyday work, close.** For long, tangled problems across many files — **no**, and we should not pretend otherwise.

Everything above is an estimate built from real measurements on similar machines. The first thing we build is the test that replaces these numbers with ours.

The bottom line

Six people. Their own terminal. A page of text every 20–30 seconds.
A genius down the hall for the hard ones.
And all of it allowed to read the data.

What we are promising

- Everyone can use it at once
- Two to three times slower than Claude
- Good enough for everyday work
- Nothing ever leaves the building
- Nobody has to babysit it

What we are not promising

- It will not match Claude on hard problems
- The specialist serves one person at a time
- It uses a real share of the lab's computers
- Every number here still needs confirming

Slower. Not as clever. Allowed in the room where the data is.

That last one is the only reason to build it — and it is enough.